

Assignment 2: Lenia — An Artificial Life

Plabon Shaha and Guillem Masdemont

University of Ljubljana, Spring Semester, EMAI Cohort 25–27

April 14, 2026

Implementation We parallelize the algorithm and present several versions that are built on top of each other. We compare the speedup performance with the baseline model. We consider the following 4 models:

- **GPU Naive:** A straightforward embarrassingly parallel CUDA implementation where each thread computes one output cell, with global memory accesses and inefficient % modulo for toroidal wrapping.
- **GPU V1 (Shared Memory):** Loads a halo-padded tile of the world into `__shared__` memory before the convolution, replacing repeated global memory reads with fast on-chip accesses.
- **GPU V2 (Constant Memory + No Modulo):** Moves the convolution kernel to `__constant__` memory for cached hardware broadcasting, and replaces modulo boundary % wrapping with cheaper `if/else` bounds checks.
- **GPU V3 (FP32):** Switches from double to float throughout, unlocking the GPU’s more numerous FP32 cores and effectively doubling usable memory bandwidth.

For the CPU parallelization part and the improved GPU versions we implemented:

- **CPU Sequential + OpenMP (evolve_lenia_seq_v2):** Refactors the original CPU baseline and parallelizes dominant loops with OpenMP. This version is used as the reference T_{seq} for speedup calculations.
- **GPU V5 (Autotuned FP32, evolve_lenia_v5):** Evaluates several CUDA thread-block geometries at startup and automatically selects the fastest configuration for the target GPU, then runs the full FP32 simulation with that tile shape.
- **GPU V6 (Dual-GPU Domain Split, evolve_lenia_v6):** Splits the world horizontally across two GPUs and exchanges halo rows every step (peer-to-peer when available, otherwise host-staged), enabling larger throughput than single-device execution.
- **GPU V4 (Kernel Fusion + Texture LUT):** Fuses the convolution and evolution steps into a single kernel pass with ping-pong buffers, eliminating the intermediate `d_tmp` write-back. The growth function is precomputed into a 1D texture object (`GROWTH_LUT_SIZE = 1024`), replacing per-thread `expf` calls with a hardware-interpolated lookup.

Results The left plot (log scale) shows the sequential CPU reaching ~ 5000 s at 4096^2 , while all GPU variants stay below ~ 3 s. The right plot expresses this as speedup over CPU, and we peak up at $7866\times$ for V4 (this result could be further improved by merging it with the 2 GPUs approach).

We decided to pursue an incremental building approach: the Naive GPU already delivers ~ 100 – $500\times$ through parallelism alone. Shared memory (V1) and constant memory with `if/else` wrapping (V2) reduce redundant loads and arithmetic overhead incrementally. Then we switched to FP32 (V3) is the single largest jump, roughly doubling bandwidth and unlocking full FP32 throughput. Finally, kernel fusion with ping-pong buffers and a texture-based growth LUT (V4) eliminates the intermediate write-back and costly `expf` calls to reach peak performance. V5 matches V3 via auto-tuned block shapes, we obtained that the best tiling performance is achieved using a 32×16 sized blocks, though the improvement is barely noticeable. Benchmarks were run over 10 simulations for the GPU variants and 5 for the CPU ones. A table of contents with the mean values and uncertainties is presented in the Appendix.

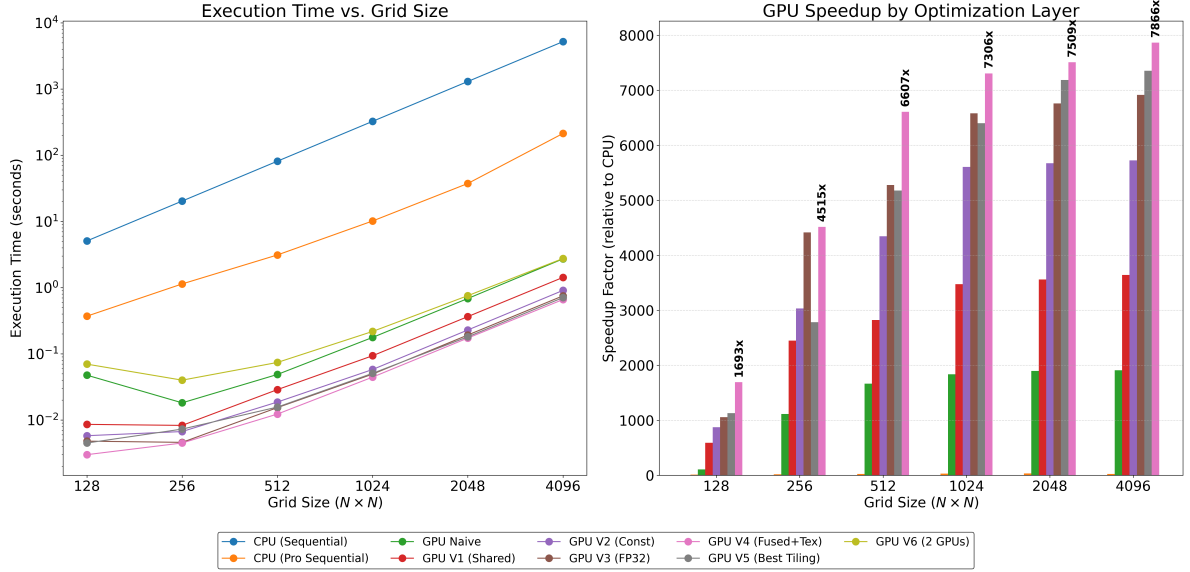


Figure 1: Performance comparison of CUDA optimization layers. **Left:** Log-log plot showing execution time versus grid size, highlighting the algebraic scaling advantage of the GPU implementations. **Right:** Grouped bar chart detailing the speedup factor of each GPU kernel relative to the sequential CPU baseline for large grid sizes.

The version V6 (2 GPUs) builds on the Naive baseline rather than V3/V4, and we split the grid horizontally across two devices. Halo-exchange overhead limits scaling, so V6 trails the single-GPU optimised variants and only converges at the largest grid sizes where compute dominates transfer cost. This could be further improved by using the techniques used for the single GPU version. Now, we plot the relative speed performance as a function of the image size and the tile size:

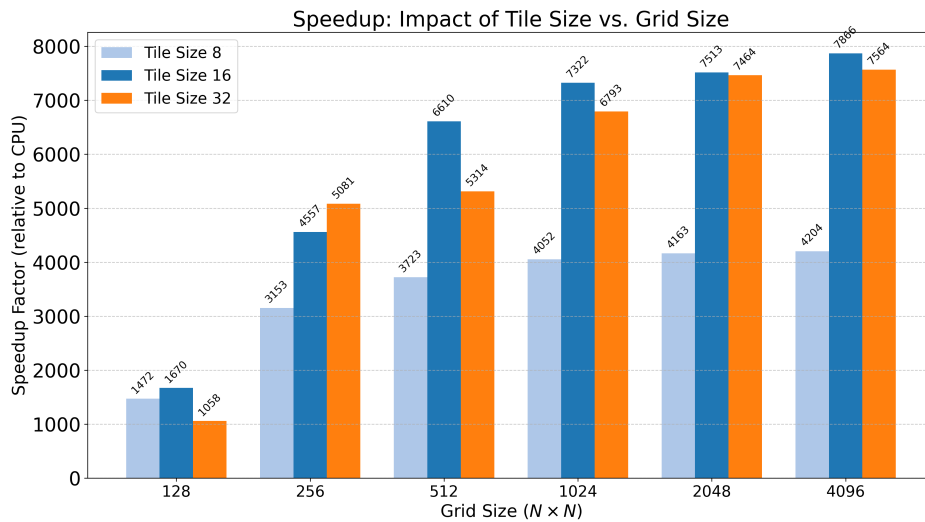


Figure 2: Comparison of the overall speedup factor across different thread block tile sizes.

The best performance is achieved using tile sizes of $N = 16$.

Appendix: Recollected data

Table 1: Performance Metrics with Uncertainty (Tile Size 8)

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	$T_{v4}(f)$	Speedup(v4)
128	5.0847	0.0465 ± 0.0983	0.0049 ± 0.0007	0.0053 ± 0.0040	0.0031 ± 0.0001	0.0035 ± 0.0009	1472.12
256	20.3207	0.0171 ± 0.0008	0.0098 ± 0.0005	0.0077 ± 0.0003	0.0065 ± 0.0003	0.0064 ± 0.0003	3153.43
512	81.3130	0.0501 ± 0.0035	0.0330 ± 0.0015	0.0283 ± 0.0095	0.0223 ± 0.0003	0.0218 ± 0.0005	3723.12
1024	325.2871	0.1750 ± 0.0014	0.1309 ± 0.0155	0.0873 ± 0.0018	0.0828 ± 0.0004	0.0803 ± 0.0009	4052.11
2048	1301.3061	0.6970 ± 0.0009	0.4971 ± 0.0011	0.3512 ± 0.0008	0.3206 ± 0.0003	0.3125 ± 0.0002	4163.95
4096	5205.5831	2.7612 ± 0.0020	1.9561 ± 0.0059	1.3998 ± 0.0025	1.2719 ± 0.0001	1.2382 ± 0.0001	4204.24

Table 2: Performance Metrics with Uncertainty (Tile Size 16)

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	$T_{v4}(f)$	Speedup(v4)
128	5.0795	0.0466 ± 0.0961	0.0094 ± 0.0152	0.0037 ± 0.0001	0.0028 ± 0.0001	0.0030 ± 0.0004	1670.89
256	20.3176	0.0170 ± 0.0008	0.0076 ± 0.0003	0.0053 ± 0.0007	0.0050 ± 0.0019	0.0045 ± 0.0001	4557.75
512	81.2717	0.0500 ± 0.0057	0.0239 ± 0.0027	0.0155 ± 0.0007	0.0128 ± 0.0013	0.0123 ± 0.0011	6610.68
1024	325.1207	0.1760 ± 0.0072	0.0890 ± 0.0061	0.0744 ± 0.0226	0.0454 ± 0.0012	0.0445 ± 0.0029	7322.62
2048	1300.6583	0.6915 ± 0.0103	0.3539 ± 0.0108	0.2934 ± 0.0207	0.1729 ± 0.0068	0.1732 ± 0.0200	7513.91
4096	5203.7136	2.7695 ± 0.0163	1.3920 ± 0.0069	1.2082 ± 0.0033	0.6839 ± 0.0028	0.6615 ± 0.0035	7866.53

Table 3: Performance Metrics with Uncertainty (Tile Size 32)

N	T_{seq}	T_{naive}	T_{v1}	T_{v2}	$T_{v3}(f)$	$T_{v4}(f)$	Speedup(v4)
128	5.1617	0.0278 ± 0.0470	0.0081 ± 0.0005	0.0053 ± 0.0003	0.0049 ± 0.0014	0.0049 ± 0.0012	1058.81
256	20.3258	0.0191 ± 0.0060	0.0077 ± 0.0015	0.0048 ± 0.0001	0.0040 ± 0.0001	0.0040 ± 0.0001	5081.45
512	81.2846	0.0478 ± 0.0019	0.0271 ± 0.0011	0.0189 ± 0.0078	0.0150 ± 0.0020	0.0153 ± 0.0046	5314.11
1024	325.0540	0.1730 ± 0.0014	0.0928 ± 0.0114	0.0585 ± 0.0114	0.0489 ± 0.0019	0.0478 ± 0.0057	6793.61
2048	1300.5371	0.6897 ± 0.0022	0.3611 ± 0.0023	0.2248 ± 0.0011	0.1843 ± 0.0014	0.1742 ± 0.0003	7464.55
4096	5205.6769	2.7613 ± 0.0105	1.4221 ± 0.0083	0.9046 ± 0.0065	0.7161 ± 0.0080	0.6881 ± 0.0083	7564.73